DESARROLLO WEB EN ENTORNO CLIENTE

PRÁCTICAS GUIADAS

OBJETOS EN NOTACIÓN LITERAL EN JAVASCRIPT

Cuatro prácticas y un reto: de las variables sueltas a los objetos literales, los métodos con this, los arrays de objetos y JSON



Realizado por Ramón Abramo

ALPE Formación • Santander

Desarrollo de Aplicaciones Web



Índice de contenidos y guía de trabajo

ESTRUCTURA DE LAS PRÁCTICAS, ITINERARIO Y ENLACES RÁPIDOS

Esta hoja de ruta reúne todo lo que vas a hacer.

Pulsa en cualquier tarjeta para saltar a su sección.

TEORÍA Y FUNDAMENTOS

Cómo trabajamos y teoría básica

Carpetas, reglas de trabajo y la sintaxis que necesitas.

Ir a la teoría (pág. 3)

PRÁCTICA 1 · CONSOLA

Primeros pasos con objetos literales

Crea, lee, modifica y borra propiedades. Añade un método.

Ir a la práctica 1 (pág. 5)

PRÁCTICA 2 · GUIADA

Ficha de personaje

Métodos con this que modifican las propiedades del propio objeto.

Ir a la práctica 2 (pág. 8)

PRÁCTICA 3 · GUIADA

Carrito de la compra

Un array de objetos literales recorrido con for... of.

Ir a la práctica 3 (pág. 11)

PRÁCTICA 4 · GUIADA

Agenda de contactos con JSON

Guarda objetos en localStorage con JSON.stringify y JSON.parse.

Ir a la práctica 4 (pág. 14)

RETO FINAL

Cuenta bancaria con saldo privado

Una clausura para que el saldo no sea accesible desde fuera.

Ir al reto (pág. 17)

FUENTES

Fuentes y Documentación Oficial

Documentación en la que se apoyan las prácticas.

Ir a las fuentes (pág. 19)



Cómo vamos a trabajar

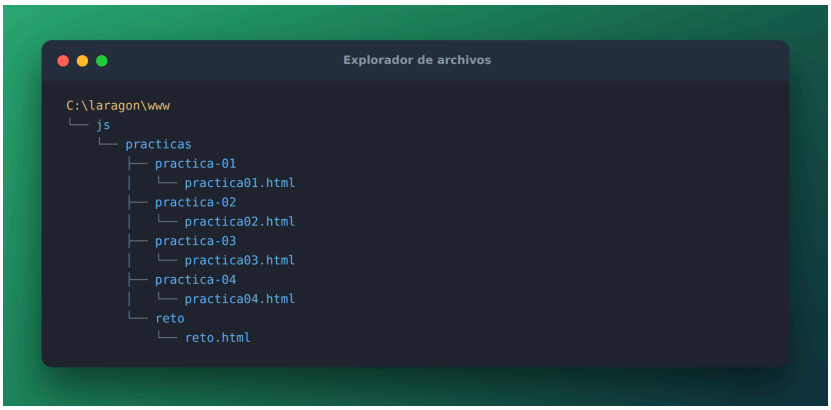
CARPETAS, ARCHIVOS BASE Y REGLAS DE TRABAJO



Dónde se guardan las prácticas

Todas las prácticas viven en Laragon, en la carpeta `C:\laragon\www\js\practicas\`.

Cada práctica tiene su propia carpeta con su archivo HTML.





Qué incluye cada práctica

- Un archivo `.html` con el **HTML y el CSS ya terminados**.
- Un **guion comentado** dentro de `<script>` con los pasos que debes completar.
- Tu trabajo es escribir el **JavaScript**: no modifiques el HTML ni los `id`.



Cómo abrir cada práctica

Todas las direcciones siguen el mismo patrón. Por ejemplo, la de la práctica 1:
`http://localhost/js/practicas/practica-01/practica01.html`

Abre la consola del navegador con `F12` y la pestaña **Consola**.



Tres reglas de trabajo

1 · Escribe tú el código

Hazlo paso a paso, siguiendo los comentarios del guion. No copies una solución.

2 · Comprueba después de cada paso

Guarda, recarga y mira la página y la consola antes de seguir.

3 · Lee los mensajes de error

La consola indica el tipo de error, el archivo y la línea.



Teoría básica

LA SINTAXIS QUE NECESITAS PARA LAS PRÁCTICAS

Un **objeto literal** agrupa datos (propiedades) y acciones (métodos) bajo un mismo nombre.

Se escribe entre llaves, con pares `clave: valor` separados por comas. [1]

Operación	Sintaxis	Ejemplo
Crear	`const obj = { clave: valor };`	`const libro = { titulo: "El Quijote" };`
Leer	`obj.clave` · `obj["clave"]`	`libro.titulo`
Modificar o añadir	`obj.clave = valor;`	`libro.leido = true;`
Borrar	`delete obj.clave;`	`delete libro.paginas;`
Comprobar	`"clave" in obj`	`"leido" in libro`
Método	`nombre() { ... }`	`saludar() { return "Hola"; }`
this	Dentro de un método, el objeto que lo llama [2]	`this.vida -= puntos;`
Array de objetos	`[ { ... }, { ... } ]`	`carrito.push(producto);`



¿Por qué usar objetos literales?

Criterio	Variables o arrays sueltos	Objetos literales
Organización	Datos dispersos que pueden desincronizarse	Todo viaja unido bajo una única entidad
Comportamiento	Funciones sueltas que reciben todos los datos	Los métodos viven en el objeto y usan `this`
Listas	Varios arrays paralelos con el mismo índice	Un único array de objetos

> **Fíjate:** esta tabla resume lo esencial. Para ampliarlo, consulta el **Manual de objetos literales en JavaScript**.



Práctica 1

CONSOLA · CREAR, LEER, MODIFICAR Y BORRAR

PRÁCTICA 1 · CONSOLA

Primeros pasos con objetos literales



Objetivo de aprendizaje

Crea un objeto literal y practica las operaciones básicas.

Lee, modifica, añade y borra propiedades.

Añade un método que use `this`.



Pasos de ejecución

1 Preparar la carpeta y el archivo

- Dentro de `C:\laragon\www\js\practicas\`, crea la carpeta `practica-01`.
- Copia en ella el archivo base `practica01.html`.
- Ábrelo con Visual Studio Code.

2 Completar el guion

- Escribe tu código dentro de `<script>`, debajo del comentario de cada paso.
- Los resultados aparecen en la consola: no hay nada que ver en la página.
- Sigue los pasos en el orden del guion.

3 Guardar, abrir y verificar

- Guarda el archivo con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre en el navegador `http://localhost/js/practicas/practica-01/practica01.html`.
- Pulsa `F12` y abre la pestaña **Consola**.
- Compara lo que ves con el resultado esperado de la página siguiente.

APOYO TEÓRICO

Objeto literal
Se escribe entre llaves, con pares `clave: valor` separados por comas. [1]

Punto y corchetes
`libro.titulo` y `libro["titulo"]` leen la misma propiedad.

Añadir y borrar
Asignar a una clave nueva la crea. `delete libro.clave` la elimina.

this
Dentro de un método, `this` es el objeto que lo llama. [2]



Práctica 1 · El guion

COMPLETA CADA PASO DEBAJO DE SU COMENTARIO

GUION DE LA PRÁCTICA 1 · practica01.html

practica01.html · <script>

```
1  // PASO 1: crea el objeto literal "libro" con estas propiedades:
2  //   titulo ("El Quijote"), autor ("Miguel de Cervantes"), paginas (863) y leido (false)
3
4
5  // PASO 2: muestra en la consola el título (notación de punto)
6  //         y el autor (notación de corchetes)
7
8
9  // PASO 3: cambia "leido" a true, añade la propiedad "genero" ("Novela")
10 //         y borra la propiedad "paginas". Muestra el objeto completo.
11
12
13 // PASO 4: comprueba con el operador "in" si "paginas" sigue existiendo
14 //         y muestra el resultado
15
16
17 // PASO 5: añade al objeto "libro" el método resumen(), que devuelve
18 //         "El Quijote (Miguel de Cervantes) - leído" o "... - pendiente"
19 //         según el valor de this.leido. Muestra lo que devuelve.
20
21
22 // PASO 6: muestra con Object.keys(libro) la lista de claves del objeto
```

Escribe tu código en los espacios en blanco, debajo de cada comentario.

Los números de línea son solo una referencia.



Práctica 1 · Comprobación

EL RESULTADO ESPERADO EN LA CONSOLA



Lo que debes ver en la consola

Consola del navegador (F12)

```
El Quijote
Miguel de Cervantes
{
  titulo: 'El Quijote',
  autor: 'Miguel de Cervantes',
  leido: true,
  genero: 'Novela'
}
false
El Quijote (Miguel de Cervantes) - leído
[ 'titulo', 'autor', 'leido', 'genero', 'resumen' ]
```

El formato de los objetos puede variar un poco entre navegadores.

Lo importante son los valores.



Errores frecuentes en esta práctica

Lo que ves	Qué debes revisar
`libro is not defined`	Declara el objeto en el paso 1, antes de usarlo en los demás.
`Unexpected identifier`	Falta una coma entre dos propiedades del objeto.
`resumen()` muestra `undefined`	Escribe `this.titulo`, no solo `titulo`. No uses una función flecha.
No sale nada en la consola	Abre `F12` y la pestaña **Consola** y recarga la página.



Práctica 2

GUIADA · MÉTODOS CON THIS

PRÁCTICA 2 · GUIADA

Ficha de personaje



Enunciado

Con variables sueltas (`nombre` , `nivel` , `vida`), cada función tendría que recibirlas y modificarlas por separado.

Aquí lo agruparás todo en un único objeto literal llamado `personaje` .

Tendrá cuatro propiedades y tres métodos que modifican sus propias propiedades con `this` .



Pasos de ejecución

1 **Preparar la carpeta y el archivo**

- Dentro de `C:\laragon\www\js\practicas\` , crea la carpeta `practica-02` .
- Copia en ella el archivo base `practica02.html` .
- Ábrelo con Visual Studio Code.

2 **Completar el guion**

- Escribe tu código dentro de `<script>` , debajo del comentario de cada paso.
- No modifiques el HTML ni los `id` .
- Sigue los pasos en el orden del guion.

3 **Guardar, abrir y verificar**

- Guarda el archivo con `Ctrl + S` .
- Comprueba en Laragon que Apache está encendido.
- Abre en el navegador `http://localhost/js/practicas/practica-02/practica02.html` .
- Pulsa los botones y comprueba que la ficha se actualiza.

APOYO TEÓRICO

Métodos con this
Un método es una función dentro del objeto. `this.vida` es la propiedad `vida` del objeto que llama al método. [2]

Escuchadores
`boton.addEventListener("click", () => { ... })` ejecuta el código en cada clic.

Pista del profesor: cada método devuelve un mensaje: es el texto que debes mostrar en `registro` .



Práctica 2 · El guion

LOS PASOS 2 A 5 ESTÁN POR COMPLETAR

GUION DE LA PRÁCTICA 2 · practica02.html

practica02.html · <script> (extracto)

```
// PASO 1 (ya hecho): constantes que apuntan a los elementos del DOM
// Constantes ya declaradas en el archivo:
//   txtNombre, txtClase, txtNivel, txtVida, registro, btnDanio, btnCurar, btnNivel

// PASO 2: crea el objeto literal "personaje" con sus propiedades y métodos
//   Propiedades: nombre ("Valeria"), clase ("Exploradora"), nivel (1), vida (100)
//   recibirDanio(puntos): resta "puntos" a this.vida (sin bajar de 0) y devuelve un mensaje
//   curar(puntos): suma "puntos" a this.vida (sin pasar de 100) y devuelve un mensaje
//   subirNivel(): suma 1 a this.nivel, restaura this.vida a 100 y devuelve un mensaje
const personaje = {

};

// PASO 3: completa la función que vuelca las propiedades del objeto en el HTML
function actualizarPantalla() {

}

// PASO 4: añade los escuchadores "click" a btnDanio (20), btnCurar (15) y btnNivel
//   En cada clic: llama al método de "personaje", muestra lo que devuelve
//   en "registro" y llama a actualizarPantalla()


// PASO 5: llama a actualizarPantalla() para pintar los datos al cargar la página
```



Los `id` del HTML que vas a usar

id	Elemento	Para qué se usa
`txtNombre`, `txtClase`	`<strong>`	Mostrar el nombre y la clase
`txtNivel`, `txtVida`	`<strong>`	Mostrar el nivel y los puntos de vida
`btnDanio`, `btnCurar`, `btnNivel`	`<button>`	Botones de acción
`registro`	`<p>`	Mensaje de la última acción



Práctica 2 · Comprobación

EL RESULTADO ESPERADO



Lo que debes ver en el navegador



AL CARGAR LA PÁGINA



TRAS DAÑO ×2 Y CURAR



Qué debes comprobar

Acción	Resultado esperado
Cargar la página	Valeria · Exploradora · nivel 1 · 100 / 100
Pulsar **Daño** dos veces	Vida 60 / 100 y mensaje «Valeria recibe 20 de daño.»
Pulsar **Curar**	Vida 75 / 100
Pulsar **Subir Nivel**	Nivel 2 y vida 100 / 100
Pulsar **Daño** muchas veces	La vida se queda en 0, nunca baja de 0



Errores frecuentes

Lo que ves	Qué debes revisar
La vida no cambia al pulsar	Falta llamar a `actualizarPantalla()` dentro del escuchador.
`NaN` en la vida	Escribe `this.vida`, no `vida`. Revisa los nombres de las propiedades.
El método da `undefined`	No uses una función flecha como método: no vincula `this`. [3]



Práctica 3

GUIADA · ARRAY DE OBJETOS LITERALES

PRÁCTICA 3 · GUIADA

Carrito de la compra



Enunciado

En lugar de tres arrays paralelos (`nombres[]` , `precios[]` y `cantidades[]`), usarás un único array llamado `carrito` .

Cada producto será un objeto literal con `nombre` , `precio` , `cantidad` y un método `subtotal()` .

Al pulsar «Añadir al carrito» crearás el objeto, lo guardarás en el array y recorrerás el array para pintar la lista y calcular el total.



Pasos de ejecución

1. **Preparar la carpeta y el archivo**
   - Dentro de `C:\laragon\www\js\practicas\` , crea la carpeta `practica-03` .
   - Copia en ella el archivo base `practica03.html` .
   - Ábrelo con Visual Studio Code.

2. **Completar el guion**
   - Escribe tu código dentro de `<script>` , debajo del comentario de cada paso.
   - No modifiques el HTML ni los `id` .
   - Sigue los pasos en el orden del guion.

3. **Guardar, abrir y verificar**
   - Guarda el archivo con `Ctrl + S` .
   - Comprueba en Laragon que Apache está encendido.
   - Abre en el navegador `http://localhost/js/practicas/practica-03/practica03.html` .
   - Añade «Teclado» a 59.90 con 2 unidades y «Ratón» a 25 con 1.
   - Comprueba que el total es 144.80 €.

APOYO TEÓRICO

Propiedades abreviadas
Si la variable se llama como la clave, basta con `{ nombre, precio, cantidad }` . [1]

Recorrer el array
`for (const prod of carrito) { ... }` te da cada objeto, uno a uno.

textContent
`textContent` escribe el texto tal cual. `innerHTML` interpreta etiquetas: no lo uses con datos que escribe el usuario.



Práctica 3 · El guion

LOS PASOS 3 Y 4 ESTÁN POR COMPLETAR

GUION DE LA PRÁCTICA 3 · practica03.html

practica03.html · <script> (extracto)

```
// PASO 1 (ya hecho): constantes que apuntan a los elementos del DOM
// Constantes ya declaradas en el archivo:
//   inpNombre, inpPrecio, inpCantidad, btnAgregar, listaCarrito, txtTotal

// PASO 2 (ya hecho): array que almacenará un objeto literal por cada producto
const carrito = [];

// PASO 3: escuchador del botón "Añadir al carrito"
btnAgregar.addEventListener("click", () => {
    // 3.1 Lee los valores de los inputs (usa Number() para precio y cantidad)

    // 3.2 Valida: el nombre no puede estar vacío y precio y cantidad deben ser mayores que
0

    // 3.3 Crea el objeto literal "nuevoProducto" con las propiedades
    //     nombre, precio, cantidad y el método subtotal()

    // 3.4 Añádelo a "carrito" con push(), limpia los inputs y llama a dibujarCarrito()

});

// PASO 4: función que recorre el array de objetos y muestra el resultado
function dibujarCarrito() {
    // 4.1 Vacía la lista (textContent = "") y crea let sumaTotal = 0

    // 4.2 Recorre "carrito" con for (const prod of carrito)

    // 4.3 Crea un <li> por producto con su nombre, sus unidades y su subtotal()

    // 4.4 Acumula en sumaTotal y muestra sumaTotal.toFixed(2) en txtTotal

}
```



Los `id` del HTML que vas a usar

id	Elemento	Para qué se usa
`inpNombre`, `inpPrecio`, `inpCantidad`	`<input>`	Datos del producto
`btnAgregar`	`<button>`	Añadir el producto al carrito
`listaCarrito`	`<ul>`	Lista de productos añadidos
`txtTotal`	`<span>`	Importe total a pagar



Práctica 3 · Comprobación

EL RESULTADO ESPERADO



Lo que debes ver en el navegador

Cesta de la Compra

Producto (ej. Tecladc | Precio (€) | 1

Añadir al carrito

Total: 0.00 €

AL CARGAR LA PÁGINA

Cesta de la Compra

Producto (ej. Tecladc | Precio (€) | 1

Añadir al carrito

Teclado mecánico (x2)	**119.80 €**
Ratón inalámbrico (x1)	**25.00 €**

Total: 144.80 €

TRAS AÑADIR DOS PRODUCTOS



Qué debes comprobar

Acción	Resultado esperado
Añadir «Teclado», 59.90 × 2	Una línea con 119.80 € y total 119.80 €
Añadir «Ratón», 25 × 1	Dos líneas y total 144.80 €
Añadir un producto sin nombre	No se añade nada
Añadir un precio 0 o negativo	No se añade nada
Tras añadir un producto	Los campos se vacían y las unidades vuelven a 1



Errores frecuentes

Lo que ves	Qué debes revisar
`NaN` en el total	Convierte precio y cantidad con `Number()`.
Los productos se duplican al añadir	Vacía la lista (`textContent = ""`) al empezar `dibujarCarrito()`.
`subtotal is not a function`	El método se llama `subtotal()` y se declara dentro del objeto.



Práctica 4

GUIADA · JSON Y LOCALSTORAGE

PRÁCTICA 4 · GUIADA

Agenda de contactos con JSON



Enunciado

Construirás una agenda que no pierde los datos al recargar la página.

Cada contacto es un objeto literal. El array de contactos se convierte a JSON y se guarda en `localStorage`.

`localStorage` solo guarda texto: por eso necesitas `JSON.stringify` y `JSON.parse`. [4] [5]



Pasos de ejecución

1 Preparar la carpeta y el archivo

- Dentro de `C:\laragon\www\js\practicas\`, crea la carpeta `practica-04`.
- Copia en ella el archivo base `practica04.html`.
- Ábrelo con Visual Studio Code.

2 Completar el guion

- Escribe tu código dentro de `<script>`, debajo del comentario de cada paso.
- No modifiques el HTML ni los `id`.
- Sigue los pasos en el orden del guion.

3 Guardar, abrir y verificar

- Guarda el archivo con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre en el navegador `http://localhost/js/practicas/practica-04/practica04.html`.
- Añade tres contactos y recarga con `F5`: siguen ahí.
- Escribe «an» en el buscador y borra un contacto.

APOYO TEÓRICO

JSON
Formato de texto para intercambiar datos. `JSON.stringify(obj)` pasa de objeto a texto; `JSON.parse(texto)` hace lo contrario. [4]

localStorage
Guarda pares clave-valor en el navegador. Las claves y los valores son siempre texto. [5]

Identificador
`Date.now()` da un número distinto en cada llamada: sirve de `id` para borrar un contacto concreto.



Práctica 4 · El guion

LOS PASOS 2 A 7 ESTÁN POR COMPLETAR

GUION DE LA PRÁCTICA 4 · practica04.html

practica04.html · <script> (extracto)

```
// PASO 1 (ya hecho): constantes que apuntan a los elementos del DOM
// Constantes ya declaradas en el archivo:
//   inpNombre, inpTelefono, btnAgregar, inpBuscar, listaContactos, txtContador

// PASO 2: clave de almacenamiento y array "contactos" (un objeto literal por contacto)
//   Lee el texto guardado con localStorage.getItem(CLAVE), conviértelo con
//   JSON.parse() y usa un array vacío [] si todavía no hay datos
const CLAVE = "agenda";
let contactos;

// PASO 3: función guardar()
//   Convierte "contactos" a texto con JSON.stringify() y guárdalo con localStorage.setItem
()


// PASO 4: función dibujar()
//   4.1 Lee el texto de búsqueda de inpBuscar (en minúsculas y sin espacios sobrantes)
//   4.2 Filtra "contactos" con filter() para quedarte con los que contienen ese texto
//   4.3 Vacía la lista y crea un <li> por contacto con su nombre, su teléfono
//       y un botón "Borrar" que elimine ese contacto por su id, guarde y vuelva a dibujar
//   4.4 Muestra en txtContador "X de Y contactos"


// PASO 5: escuchador de btnAgregar
//   Valida nombre y teléfono, crea el objeto { id: Date.now(), nombre, telefono },
//   añádelo a "contactos", guarda, limpia los inputs y dibuja


// PASO 6: escuchador "input" de inpBuscar que llame a dibujar()


// PASO 7: llama a dibujar() para pintar los datos al cargar la página
```



Los `id` del HTML que vas a usar

id	Elemento	Para qué se usa
`inpNombre`, `inpTelefono`	`<input>`	Datos del contacto
`btnAgregar`	`<button>`	Añadir el contacto
`inpBuscar`	`<input>`	Filtrar por nombre mientras escribes
`listaContactos`, `txtContador`	`<ul>`, `<p>`	Lista y contador de contactos



Práctica 4 · Comprobación

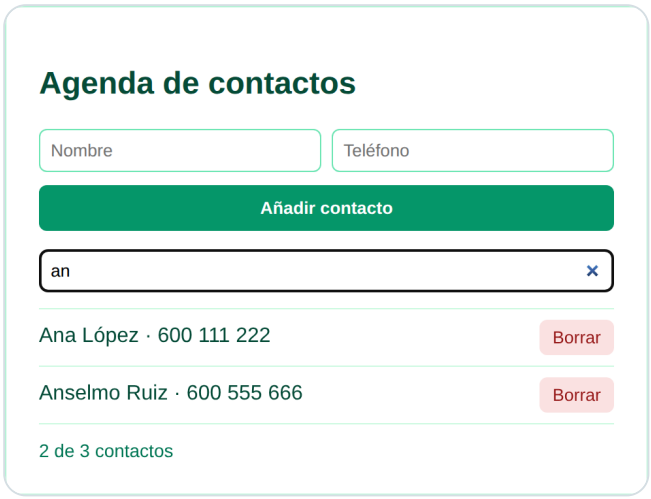
EL RESULTADO Y LOS DATOS GUARDADOS



Lo que debes ver en el navegador



TRAS AÑADIR TRES CONTACTOS



BUSCANDO «an»



Lo que se guarda en el navegador

Abre `F12` → **Aplicación** → **Almacenamiento local** y busca la clave `agenda`.

Verás un texto JSON como este (los `id` serán distintos):





Errores frecuentes

Lo que ves	Qué debes revisar
`contactos` es `null` al empezar	`JSON.parse(null)` devuelve `null`: añade `|| []` al final.
Al recargar se pierde todo	Llama a `guardar()` después de añadir y de borrar.
`Unexpected token` al arrancar	El texto guardado no es JSON válido: ejecuta `localStorage.clear()` en la consola.



Reto final

CLAUSURAS Y PRIVACIDAD EN OBJETOS LITERALES

RETO FINAL

Cuenta bancaria con saldo privado



Enunciado

Crea una cuenta bancaria cuyo saldo **no se pueda leer ni modificar desde fuera**.

En un objeto literal todo es público, así que necesitas una **clausura**: una función autoejecutable que devuelve el objeto con solo los miembros públicos.



Qué debe tener el objeto

Miembro	Qué hace
`titular`	Propiedad pública con el valor `"Lucía Navarro"`
`ingresar(importe)`	Suma al saldo y devuelve `"Saldo: 300€"`. Rechaza importes menores o iguales que 0
`retirar(importe)`	Resta del saldo. Devuelve `"Operación denegada"` si el importe no es válido o supera el saldo
`consultarSaldo()`	Devuelve el saldo actual
`historial()`	Devuelve un array con un texto por cada movimiento



Pasos de ejecución

1. **Preparar la carpeta y el archivo**
   - Dentro de `C:\laragon\www\js\practicas\`, crea la carpeta `reto`.
   - Copia en ella el archivo base `reto.html` y ábrelo con Visual Studio Code.
2. **Completar el reto**
   - Escribe el objeto `cuenta` donde indica el comentario.
   - No modifiques las pruebas del final.
3. **Guardar, abrir y verificar**
   - Guarda con `Ctrl + S` y abre `http://localhost/js/practicas/reto/reto.html`.
   - Pulsa `F12`, abre la **Consola** y compara con el resultado esperado.



Reto final · El guion y el resultado

COMPLETA EL OBJETO Y COMPARA LA CONSOLA

GUION DEL RETO · reto.html

reto.html · <script>

```
// RETO: cuenta bancaria con saldo privado
// Crea con una clausura (función autoejecutable) el objeto "cuenta":
//   - titular: "Lucía Navarro" (propiedad pública)
//   - ingresar(importe): suma al saldo y devuelve "Saldo: 300€" (rechaza importes <= 0)
//   - retirar(importe): resta del saldo y devuelve "Saldo: 200€"
//                       (devuelve "Operación denegada" si el importe es <= 0 o supera el saldo)
//   - consultarSaldo(): devuelve el saldo actual
//   - historial(): devuelve un array con un texto por movimiento
//   El saldo NO puede ser accesible desde fuera: cuenta.saldo debe dar undefined

const cuenta =

// Pruebas (no las modifiques)
console.log(cuenta.ingresar(300));
console.log(cuenta.retirar(100));
console.log(cuenta.retirar(900));
console.log(cuenta.saldo);
console.log(cuenta.consultarSaldo());
console.log(cuenta.historial());
```



Lo que debes ver en la consola

Consola del navegador (F12)

```
Saldo: 300€
Saldo: 200€
Operación denegada
undefined
200
[ 'ingreso 300€ → 300€', 'retirada 100€ → 200€' ]
```

Pista del profesor: `cuenta.saldo` debe dar `undefined`: la variable `saldo` vive dentro de la clausura y el objeto devuelto no la expone.



Fuentes y Documentación Oficial

REFERENCIAS DE LAS PRÁCTICAS

Cada número entre corchetes remite a la documentación correspondiente.



Fuentes y Documentación Oficial

[1] Trabajando con objetos · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Guide/Trabajando_con_objectos

[2] this · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Referencia/Operadores/this

[3] Funciones flecha · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Reference/Functions/Arrow_functions

[4] JSON · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/JSON

[5] Window.localStorage · MDN Web Docs · developer.mozilla.org/es/docs/Web/API/Window/localStorage